

Arrays

- ① Linear Search : It is used for searching an element in an array.
 It Can be used for both Sorted & unsorted array.
 For unsorted array this method is the only way to search an element.

Algorithm:

```

int find (int a[], int n, int key) {
    int i = 0;
    while (i < n) {
        if (a[i] == key) {
            return i;
        }
        i++;
    }
    return -1;
}

```

→ This Condition is executed/checked (n+1) times

→ this condition is checked "n" times.

- ② Binary Search : Used for searching elements in an array.
 ↳ Better than linear search.
 ↳ Can be used only for sorted arrays.

Algorithm

→
 Next page

Text Book: CLRS

Cormen, Leiserson, Rivest, Stein

09/01/2026

```

Search(int a[], int lenn, int key) {
    i = 0 ;  $\rightarrow$  time taken =  $c_1$  (constant)  $\times 1$ 
    j = lenn ;  $\rightarrow$  " =  $c_2 (n) \times 1$ 
    (A)  $\leftarrow$  while (i < j) {  $\rightarrow$  "  $\leq c_3 (n) \times (\log_2 n + 2)$ 
        mid =  $\left\lfloor \frac{i+j}{2} \right\rfloor$  ;  $\rightarrow$  " =  $c_4 (n) \times (\log_2 n + 1)$ 
        if (a[mid] == key) {  $\rightarrow$  " =  $c_5 (\log_2 n + 1)$ 
            return mid;
        }
        else if (a[mid] > key) {  $\rightarrow$  " =  $c_6 (\log_2 n + 1)$ 
            (B)  $\leftarrow$  j = mid ;
        }
        else {
            (C)  $\leftarrow$  i = mid + 1 ;
        }
    }
    return -1;
}

```

* Let x_p be the value which denotes the value of $(j-i)$ when statement A is executed for the p^{th} time.

$$x_1 = n$$

Claim: $x_{p+1} \leq \frac{x_p}{2}$, $\forall p$.

Proof: let i_p, j_p be values of i & j when statement A is executed for p^{th} time.

$$x_p = j_p - i_p$$

Case 1: Statement B got executed during p^{th} iteration of while loop.

$$j_{p+1} = (\text{mid})_p = \left\lfloor \frac{i_p + j_p}{2} \right\rfloor ; i_{p+1} = i_p$$

$$x_{p+1} = j_{p+1} - i_{p+1} = \left\lfloor \frac{i_p + j_p}{2} \right\rfloor - i_p$$

$$x_{p+1} \leq \frac{i_p + j_p}{2} - i_p$$

$$x_{p+1} \leq \frac{j_p - i_p}{2}$$

$$x_{p+1} \leq \frac{x_p}{2}$$

Case 2: If statement C gets executed in p^{th} iteration of while loop.

$$j_{p+1} = j_p$$

$$i_{p+1} = (\text{mid})_{p+1} = \left\lfloor \frac{j_p + i_p}{2} \right\rfloor + 1$$

$$x_{p+1} = j_{p+1} - i_{p+1}$$

$$x_{p+1} = j_p - \left(\left\lfloor \frac{j_p + i_p}{2} \right\rfloor + 1 \right) = j_p - \left\lfloor \frac{j_p + i_p}{2} \right\rfloor - 1$$

$$\text{As, } \left\lfloor \frac{j_p + i_p}{2} \right\rfloor \leq \frac{j_p + i_p}{2} - 1$$

$$x_{p+1} \leq j_p - \left(\frac{j_p + i_p}{2} - 1 \right) - 1$$

$$x_{p+1} \leq \frac{j_p - i_p}{2}$$

$$x_{p+1} \leq \frac{x_p}{2}$$

Hence our claim is proved in both cases.

* $x_1 = n$

$$x_2 \leq \frac{x_1}{2} \leq \frac{n}{2}$$

$$x_3 \leq \frac{x_2}{2} \leq \frac{n}{4}$$

$\vdots$

$$x_t \leq \frac{n}{2^{t-1}}$$

$$\rightarrow \text{if } t \geq \log_2 n + 1$$

$$x_t \leq \frac{n}{2^{\log_2 n}} = 1$$

$$x_t \leq 1$$

$\therefore$ Statement (A) gets executed for atmost $\log_2 n + 2$ times.

* Time Complexity

* We will observe worst case "time Complexity".



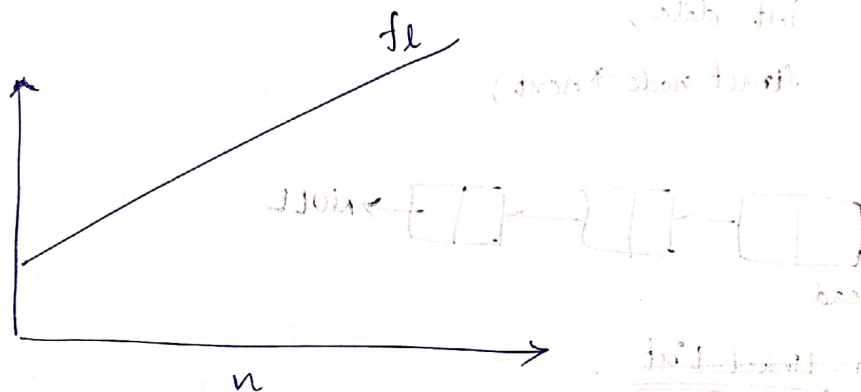
* Total time taken for ~~log~~ Algorithm is (T). [BINARY SEARCH]

$$T \leq C_1 + C_2 + (\log_2 n + 2) [C_3 + C_4 + C_5 + \dots]$$

$$T \leq \overbrace{C + D}^{f_b} (\log_2 n)$$

* Total time taken by linear search Algorithm (T').

$$T' \leq \overbrace{C' + nD'}^{f_l}$$



* struct customer of

int key;

};

struct customer a[100];

↳ We can get access to information of customer by $a[i].key$ etc.

↳ We can compare keys of two customers by

$$a[i].key < a[j].key$$

* if we are using pointer; struct customer *a[100].

$$a[0] = (\text{struct customer} *) \text{malloc}(\text{sizeof}(\text{customer})).$$

→ If we are using pointers as above, we can get access by

$$a[i] \rightarrow \text{key}.$$

For lab: Inserting & Deleting an element in an array, struct usage,

For Lab - Linked Lists, Check Stack & Queue, classes (including init),
check the working of init function,

LINKED LISTS :

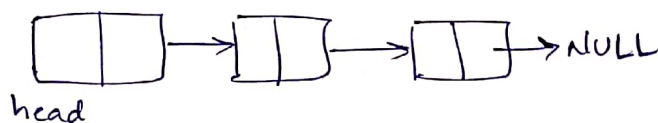
* struct node;

struct node {

int data;

struct node *next;

}



Insertion in Linked List

ABSTRACT DATA TYPES (ADT):

* STACK :

↳ Maintains "Last in First Out" policy.

Operations :

stack s;

Push (s, a);

Pop (s); → This "pop" returns the value that is pushed last.

* QUEUE :

↳ Maintains "First in First Out" policy.

Operations

queue q;

Enqueue (q, a)

Dequeue (q)

STACK

~~Stack S;~~
~~Push(S, a)~~

struct Stack {

int data[100];

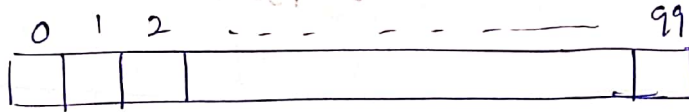
int top;

}

Stack S;

Push(S, a)

Stack



* Push Function of Stack

Stack *S;

Push(S, a)

$S \rightarrow \text{data}[S \rightarrow \text{top}] = a;$ To be done only if stack is not full.

$S \rightarrow \text{top}++;$

* isempt(S) {

if (S.top == 0) {

return true;

}

else {

return false;

}

* Note: In C-language we have to use pointer for Stack.

For calling push function/pop function use below notation,

Push(&S, a) pop(&S)

where "S" is a Stack

POP Function
~~struct stack *s~~
 pop(&s) {

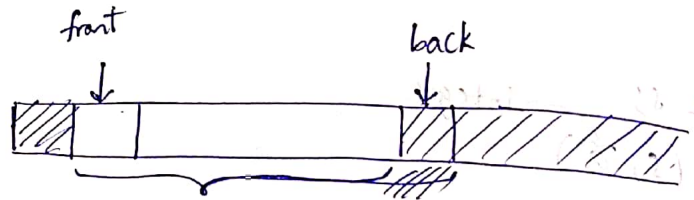
~~s = top;~~ s → top --;
 return s → data[s → top];

}

QUEUE :

struct queue {

int data[100];



~~Enqueue(Q, a) {
 Q → data[Q → back] = a;
 Q → back ++;
 }~~

~~Dequeue(Q) {
 // if queue is not empty
 Q → front ++;
 return Q → data[Q → front];
 }~~

Enqueue(Q, a) {

if (is full(Q) == false) {

Q → data[Q → back] = a;
 Q → back ++;

if (Q → back == 100) {

Q → back = 0;

}

is full(Q) {

if (Q → back == 100) {

return true;

}

else {

return false;

}

is empty(Q) {

if (Q → back == Q → front) {

return true;

}

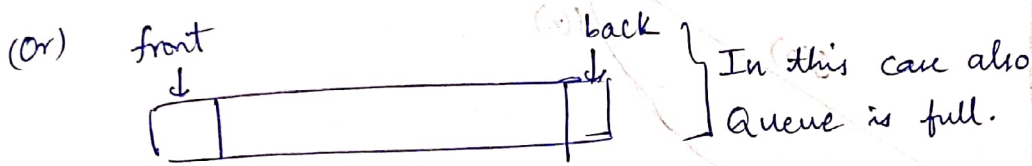
else {

return false;

}

* Queue is full if the only unused location in the array is $Q.back$.

* In general if $back+1 == front$ (or) $front-1 == back$, Queue is full.



Dequeue(Q) {

// if Q is not empty

temp = $Q.data[Q.front]$

$Q.front++$;

if ($Q.front == 100$) {

$Q.front = 0$;

}

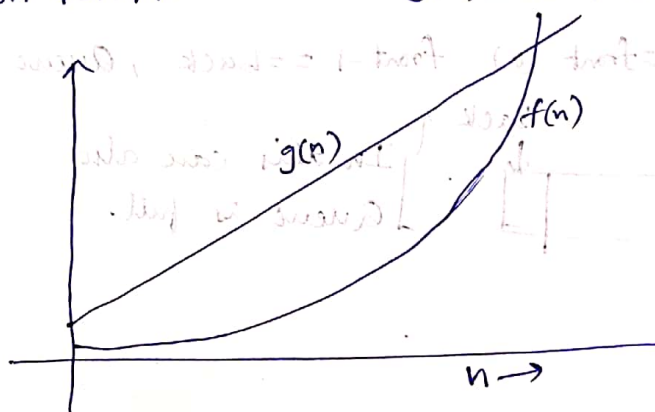
return temp;

}

Asymptotic Analysis of Functions:

$$f(n) = 3n^2 + 2n + 10$$

$$g(n) = 10n + 15$$



BIG-O Notation:

$$O(g(n)) = \{f(n) : \exists c, n_0 \text{ such that } \forall n \geq n_0, f(n) \leq c \cdot g(n)\}$$

$$\rightarrow f(n) \in O(g(n))$$

$$\text{If } f(n) = 3n^2 + 2n + 10, g(n) = 10n + 15$$

* Is $f(n) \in O(g(n))$? No.

Sol No. Because,

As per $O(g(n))$ definition,

we

$$3n^2 + 2n + 10 \leq 10cn + 15c$$

$$3n^2 + 2n + 10 - 10cn - 15c \leq 0$$

↳ This function goes to +ve infinity as n increases.

∴ inequality don't hold $\forall n$.

∴ $f(n)$ does not belongs to $O(g(n))$.

* Is $g(n) \in O(f(n))$? Yes.

$$g(n) \leq c \cdot f(n), \text{ let } c=1$$

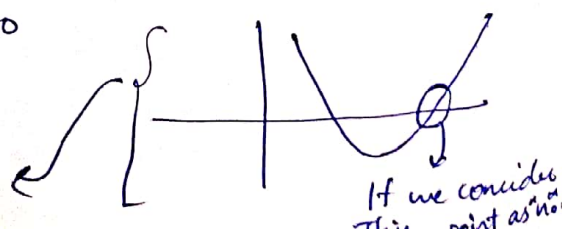
$$10n + 15 \leq 3n^2 + 2n + 10$$

$$3n^2 + 2n + 10 - 10n - 15 \geq 0$$

$$3n^2 - 8n - 5 \geq 0$$

From graph we found n_0, c .

$$\therefore g(n) \in O(f(n)).$$



If we consider n_0 as n_0 .

ex: $f(n) = 10n + 3$, $g(n) = n + 1$

Is $g(n) \in O(f(n))$? Yes

$$g(n) \leq c \cdot f(n)$$

$$n + 1 \leq 10cn + 3c \quad (10c-1)n + (3c-1) \geq 0$$

$$(10c-1)n + (3c-1) \geq 0$$

let $c=1, n_0=1$ then $\forall n \geq n_0, g(n) \leq c \cdot f(n)$

$\therefore g(n) \in O(f(n))$

Is $f(n) \in O(g(n))$? Yes

$$f(n) \leq c \cdot g(n)$$

$$10n + 3 \leq c \cdot n + c$$

10-

$$(c-10)n + (c-3) \geq 0$$

let, $c=11, n_0=10$

$\therefore f(n) \in O(g(n))$

* $\Omega(g(n)) = \{f(n) : \exists c, n_0 \text{ such that } \forall n \geq n_0, c \cdot g(n) \leq f(n)\}$

* $f(n) \in O(g(n)) \implies g(n) \in \Omega(f(n))$?

$\exists c, n_0$ such that

$\forall n \geq n_0,$

$$f(n) \leq c \cdot g(n)$$

$$\frac{1}{c} f(n) \leq g(n)$$

$\exists c, n_0$ such that

$\forall n \geq n_0$

$$g(n) \geq \frac{1}{c} f(n)$$

$$c' f(n) \leq g(n)$$

Theta Notation:

Theta: Θ

* $f(n) \in \Theta(g(n))$,

If $f(n) \in O(g(n))$ and $g(n) \in O(f(n))$

SMALL-O Notation:

$$o(g(n)) = \{f(n) : \forall c, \exists n_0 \text{ such that } \forall n \geq n_0, f(n) < c \cdot g(n)\}$$

ex: $f(n) = 10n, g(n) = n^2 + 3$

Is $f(n) \in o(g(n))$?

$$10n < c \cdot (n^2 + 3)$$

$$cn^2 - 10n + 3c > 0$$

We can take any value of 'c', but there exists a value for n_0 such that $cn^2 - 10n + 3c > 0$.

ω - Notation:

~~$f(n) \in g(n)$~~

~~$f(n) \in \omega(g(n))$~~

~~$\omega(g(n)) = \{f(n)\}$~~

$$\omega(g(n)) = \{f(n) : \forall c, \exists n_0 \text{ such that } c \cdot g(n) < f(n)\}$$

* $O(g(n)) = \{f(n) :$

$\Omega(g(n)) = \{f(n) :$

$o(g(n)) = \{f(n) :$

$\omega(g(n)) = \{f(n) :$

* If $f(n) \in O(1)$

* If $f(n) \in o(1)$

* $f(n) = n^{0.1} \Rightarrow$

* $f(n) = \log n \Rightarrow$

* $f(n) = O(g(n))$

$2n^2 + O(n) = O(n^2)$

* If $n^d \in O(n^k)$

To show $\forall c,$

* $O(O(f(n))) = O(f(n))$

$g(n) \in O(f(n))$

$\exists c_0, n_0$

$\exists c, n_1$

~~$\exists c, n_2$~~

Let

$\forall n$

$$* O(g(n)) = \{f(n) : \exists c, n_0 \text{ such that } \forall n \geq n_0, f(n) \leq c \cdot g(n)\}$$

$$\Omega(g(n)) = \{f(n) : \exists c, n_0 \text{ such that } \forall n \geq n_0, c \cdot g(n) \leq f(n)\}$$

$$o(g(n)) = \{f(n) : \forall c \exists n_0 \text{ such that } \forall n \geq n_0, f(n) < c \cdot g(n)\}$$

$$\omega(g(n)) = \{f(n) : \forall c \exists n_0 \text{ such that } \forall n \geq n_0, c \cdot g(n) < f(n)\}$$

* If $f(n) \in O(1)$, $f(n) = \frac{10}{n}$ is a possibility.

* If $f(n) \in o(1)$, $f(n) = \frac{1}{n}$

* $f(n) = n^{0.1} \Rightarrow \omega(1)$

* $f(n) = \log n \Rightarrow \omega(1)$

* $f(n) = O(g(n))$ | $O(n) = O(n^2) \checkmark$

$2n^2 + O(n) = O(n^2)$ | $O(n^2) = O(n) \times$

* If $n^d \in O(n^{d'})$, if $d' > d$

To show $\forall c, \exists n_0$ such that $\forall n \geq n_0, n^d < c \cdot n^{d'}$

$$c \cdot n^{d'} \geq n^d \Rightarrow c \cdot n^{d'-d} \geq 1$$

$$c \cdot n^{d'-d} > 1$$

$$* O(O(f(n))) = O(f(n))$$

$$g(n) \in O(f(n)), h(n) \in O(g(n)) \Rightarrow h(n) \in O(f(n)).$$

$$\exists c_0, n_0 \text{ such that } \forall n \geq n_0, g(n) \leq c_0 \cdot f(n).$$

$$\exists c_1, n_1 \text{ such that } \forall n \geq n_1, h(n) \leq c_1 \cdot g(n)$$

$$\exists c_2, n_2 \text{ such that } \forall n \geq n_2, h(n) \leq c_2 \cdot f(n)$$

$$\text{Let } c_2 = c_0 c_1, n_2 = \max\{n_0, n_1\},$$

$$\forall n \geq n_2, g(n) \leq c_0 \cdot f(n) \Rightarrow h(n) \leq c_0 \cdot c_1 \cdot f(n)$$

$$h(n) \leq c_1 \cdot g(n) \Rightarrow h(n) \leq c_2 \cdot f(n)$$

- * $O(f(n)) + O(f(n)) = O(f(n))$
- * $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$
- * $O(f(n)) \times O(g(n)) = O(f(n) \times g(n))$
- * $O(f(n)) \times \text{constant} = O(f(n))$
- * $f(n) = \log_2 n$
- $f(n) \in O(\log_2 n)$

$$\log_{10} n = \frac{\log_2 n}{\log_2 10}$$

$\therefore$ We can directly say, $f(n) \in O(\log_2 n)$

base here is not important.

- * $n \in O(1.1^n)$

$$1.1 = 1 + 0.1$$

$$(1+a)^n = \binom{n}{0} + \binom{n}{1}a + \binom{n}{2}a^2 + \dots + \binom{n}{n}a^n$$

- * $n^d \in O(2^n)$, $n^d \in O(1.1^n)$

- * By $\binom{n}{d} \geq \left(\frac{n}{d}\right)^d$

SORTING

① Insertion Sort:

$\hookrightarrow$ First "i" elements were in sorted order. We are observing from $(i+1)^{\text{th}}$ element

$$i = 1;$$

$$\text{while}(i < n) \{$$

$$j = i + 1;$$

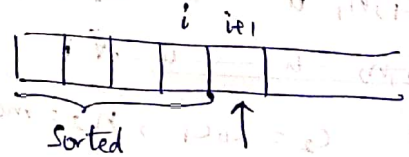
$$\text{while}(j > 0 \text{ \& } a[j] < a[j-1]) \{$$

$$\text{swap}(a[j], a[j-1]);$$

$$j = j - 1;$$

$$i = i + 1;$$

}

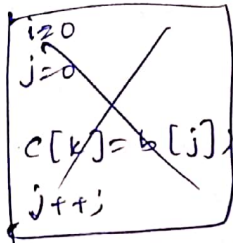


$\hookrightarrow$ Running time is

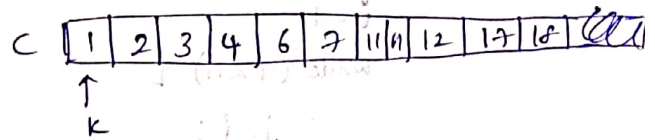
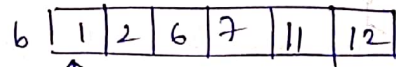
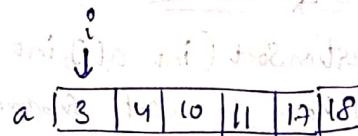
$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} = \Theta(n^2) \rightarrow \text{Worst case}$$

② Divide and Conquer [Merge Sort]: $\rightarrow O(n)$

We need to combine two arrays which were already sorted into a single sorted array.



merge(a, lena, b, lenb) {
 int i = 0
 int j = 0



③ Merge Sort:

mergeSort(a, len) { $\rightarrow$ Sort the subarray a[p], a[q-1].

mergeSort(a, p, q) {

$$\text{mid} = \frac{p+q}{2};$$

mergeSort(a, p, mid);

mergeSort(a, mid, q);

merge(a, p, mid, q);

$O(n \log n)$

INSERTION SORT

* insertionSort(int a[], int p, int q) {
 // have to sort subarray from p to (q-1).

* insertionSort(int a[], int n) {

```

  i = 1;
  while (i < n) {
    j = i;
    while (j > 0 & a[j-1] > a[j]) {
      swap(a[j-1], a[j]);
      j--;
    }
  }

```

```

* i = 1;
  while (i < n) {
    temp = a[i];
    j = i - 1;
    while (j >= 0 & a[j] > temp) {
      a[j+1] = a[j];
      j--;
    }
    a[j+1] = temp;
    i++;
  }

```

Code-1
 Insertion Sort

* If we have to sort the subarray from index p to q (not including qth element).

```

  i = p + 1;
  while (i < q) {
    temp = arr[i];
    j = i - 1;
    while (j >= p & arr[j] > temp) {
      arr[j+1] = arr[j];
      j--;
    }
    arr[j+1] = temp;
    i++;
  }

```

Loop Invariant (For Code-1); [holds at beginning of outer loop]

* $a[0], a[1], \dots, a[i-1]$ are first i elements of the input array in sorted order.

* Invariant holds for the first iteration, i.e., when $i=1$. Suppose that the invariant is true at beginning of the t^{th} iteration of the loop.

$$i = t$$

$a[0], \dots, a[t-1]$ are first t elements of input array, in sorted order.

To show: After t^{th} iteration

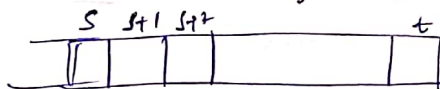
(i) $a[0], a[1], \dots, a[t]$ are first $(t+1)$ elements of input array.

j varies from $(t-1)$ to 0 .

elements $a[t+1]$ to $a[n]$ are unaffected.

(ii) $a[0], a[1], a[2], \dots, a[t]$ are in sorted order.

$$\text{let } S = \max_{r < t} \{a[r] \leq a[t]\}$$



$$a[S] \leq \text{temp}, a[S+2] > \text{temp}$$

temp is going to be stored at $(S+1)^{\text{th}}$ index.

$\Rightarrow$ Order b/w $a[0], \dots, a[t]$ is not changed.

* Time spent in the t^{th} iteration of the outer loop in worst case is $O(t)$.

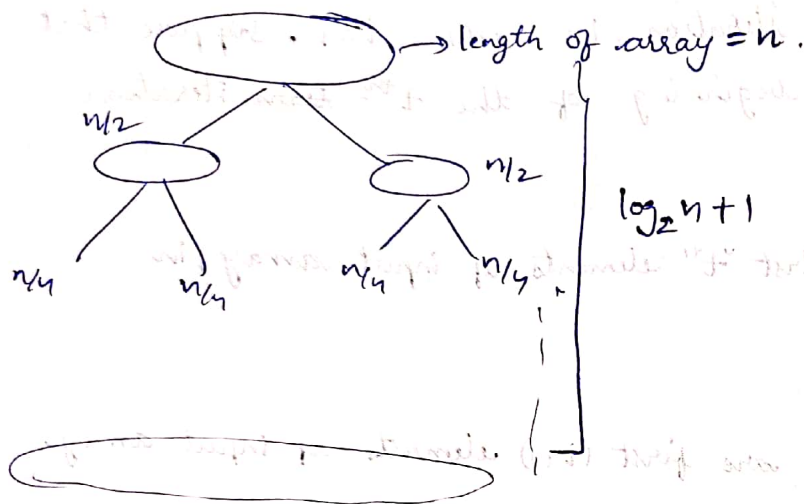
$$\sum_{t=1}^n (ct+d) \Rightarrow O(n^2)$$

$$nd + c \cdot \frac{n(n+1)}{2}$$

Merge Sort:

→ Merge sort time $T(n)$ to sort an array of n elements

$$T(n) \leq 2T(n/2) + O(n)$$

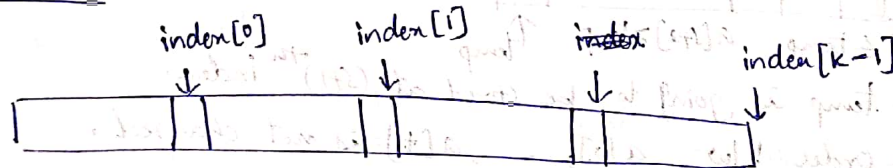


* Time Spent = $c \times n$ for one array.

$$\Rightarrow \text{Total time} = cn(\log_2 n + 1) = cn \log_2 n + cn \Rightarrow O(n \log n) + O(n) = O(n \log n).$$

28/01/2026

k-Way Merge

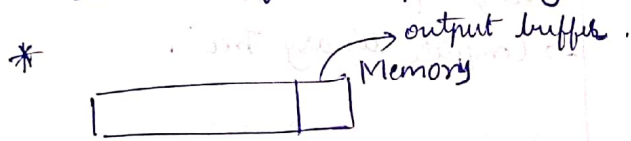


* k-blocks of size $\frac{n}{k}$

* Merge blocks pairwise, then Time complexity is $O(n \log k)$.

External Sorting: [Check This Sorting Algorithm & Code].

- * If memory size is smaller than array to be sorted. Take small blocks of input array of size of memory, then sort that memory. & put it back in array. Then we will have some blocks of sorted arrays in input array.



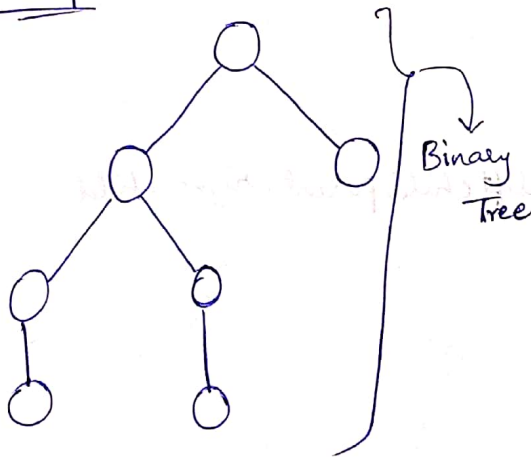
keep these small parts of input array in memory, & find smallest element in memory & put it in output buffer.

PRIORITY QUEUE:

Insert(Q, val)

Extract-min(Q) → returns the smallest number and removes it.

HEAP:



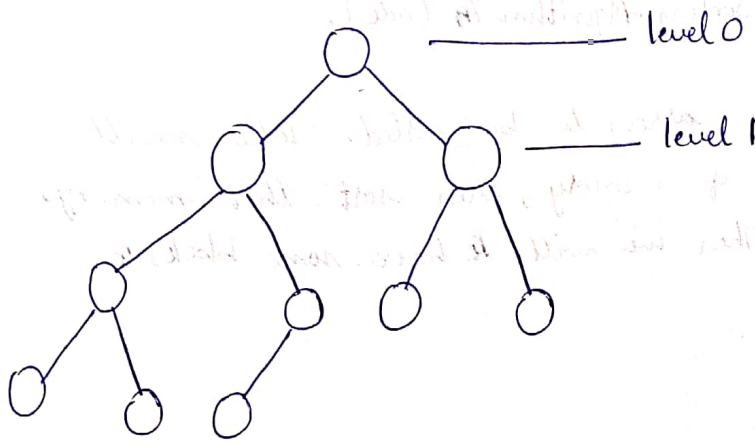
k-ary Tree
2-ary Tree = binary Tree.

Complete Binary Tree:

Every level other than the last level is full and the last level is

~~filled~~ complete from left or has no gaps. ^{complete}

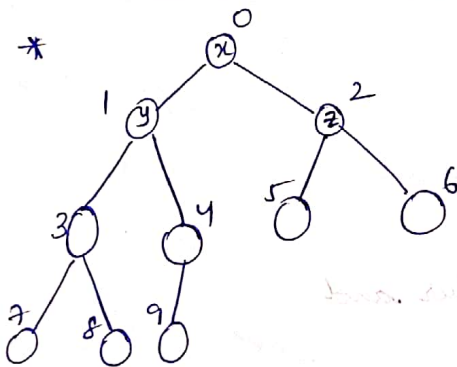
- * Any subtrees of complete binary Tree are also binary Trees.



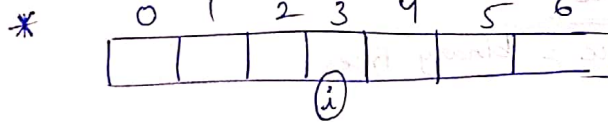
Note:
We are using
Complete Binary Tree.
→ For Complete binary Tree,
We can represent Tree as an
array of elements.
→ Complete binary Tree.

Complete Binary Tree

* i^{th} level nodes $\leq 2^i$



0	1	2	3	4	5	6	7	8	9
x	y	z							

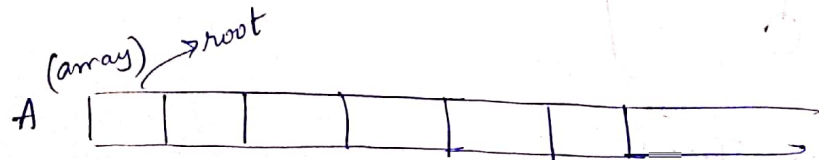


Parent(i) = $\left\lfloor \frac{i-1}{2} \right\rfloor$ } Index 'i' is left child, parent, right child.

left child(i) = $2i+1$

right child(i) = $2i+2$

MIN-HEAP:



* $A[i] \geq A[\text{parent}(i)]$

* In Min-heap, value at root is the smallest value in min-heap.

Heapify (array A, i) {

compare $A[i]$, $A[\text{leftchild}(i)]$, $A[\text{rightchild}(i)]$

~~Smallest among them~~

if smallest is $A[i]$, return

else: exchange $A[i]$ with smallest, call Heapify (A , smallest)

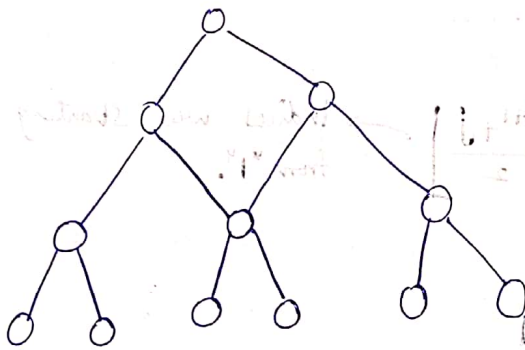
}

Build-Heap (unsorted array A) {

}

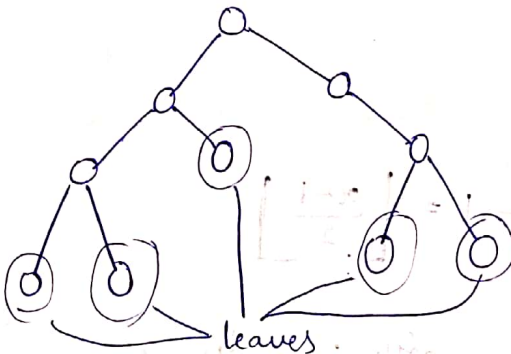
30/01/26

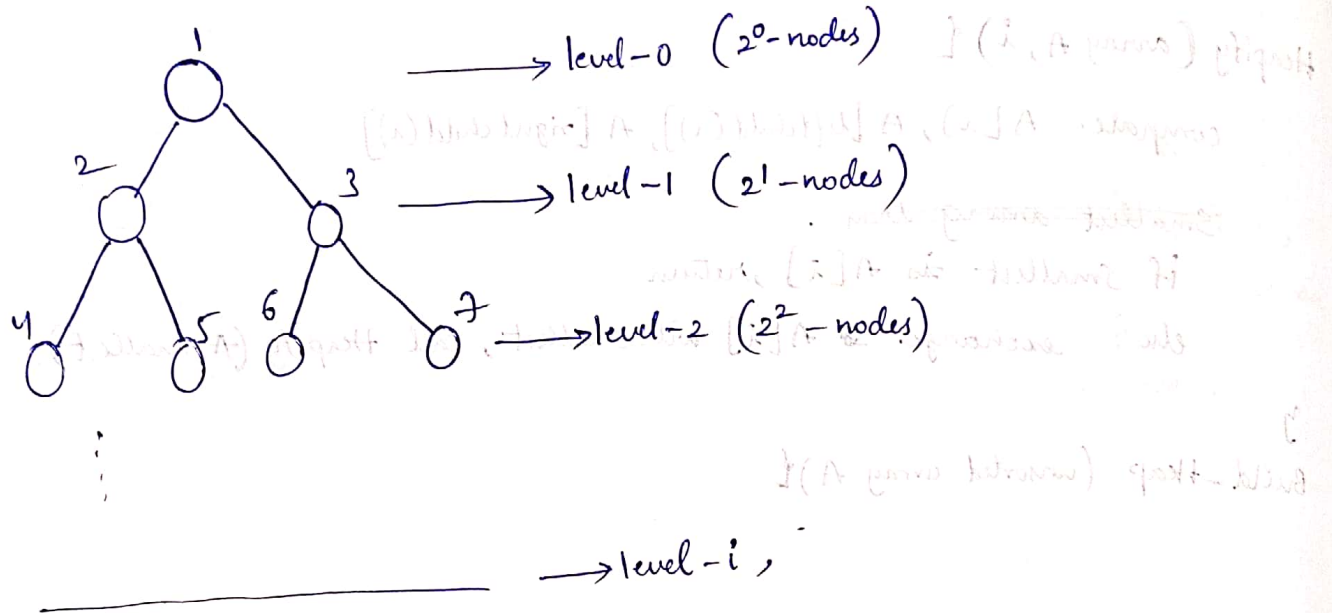
*



Not a Binary Tree

*





* For level- i of a binary tree indices will belong to $2^i, 2^i+1, 2^i+2, \dots, 2^i+2^i-1 \Rightarrow [2^i, 2^i+2^i-1]$

* For level- $(i+1)$ of a binary tree indices will be

$$\boxed{2^{i+1}, 2^{i+1}+1}, \boxed{2^{i+1}+2, 2^{i+1}+3}, \dots$$

children of 2^i children of 2^i+1

* $\text{Parent}(2^{i+1}+j) = 2^i + \left\lfloor \frac{j}{2} \right\rfloor = \left\lfloor \frac{2^{i+1}+j}{2} \right\rfloor$ → Indices will start from "1".

* If $x, y \in \mathbb{R}$, then

$$\lfloor x \rfloor + \lfloor y \rfloor \leq \lfloor x+y \rfloor$$

* If indices start from "1",

$$\text{Parent}(x) = \left\lfloor \frac{x}{2} \right\rfloor$$

* If indices start from "0",

$$\text{Parent}(x) = \left\lfloor \frac{x+1}{2} \right\rfloor - 1 = \left\lfloor \frac{x-1}{2} \right\rfloor$$

* children of node with index 2^i+j are $2^{i+1}+2j$ & $2^{i+1}+2j+1$

* If indices starting from 1,

$$\text{left}(x) = 2x$$

$$\text{right}(x) = 2x+1$$

* If indices starting from 0,

$$\text{left}(x) = 2(x+1)-1 = 2x+1$$

$$\text{right}(x) = 2(x+1)+1-1 = 2x+2$$

HEIGHT OF A NODE x :

* height of node x = Maximum length of a path from x to a leaf.

Heapify(x) [subtree of a complete binary tree are

[comparison between the values in x , $\text{left}(x)$, $\text{right}(x)$]

* If smallest = x , return

else

exchange values of x & smallest.

Heapify(smallest).

* Time taken for Heapify(x) = $c \cdot h(x) = O(h(x)) = O(\log n)$

* Total no of nodes in a binary tree with all levels were filled

is, no of nodes = $2^{i+1}-1$

Build-Heap(A): $\rightarrow$ Time Complexity : $O(n)$

for ($i = \lfloor \frac{n}{2} \rfloor - 1$ to 1) {

 Heapify(i); $\rightarrow O(\log n)$

}

* parent($n-1$) = $\lfloor \frac{n-2}{2} \rfloor$

as its Time complexity is $O(h(i))$,
at start heights were very small they take
very less time.

* No. of vertices of height 0 = $\lfloor \frac{n-2}{2} \rfloor + 1, \dots, n-1$

* No of vertices of height 1

parent($\lfloor \frac{n-2}{2} \rfloor$) = $\lfloor \frac{\lfloor \frac{n-2}{2} \rfloor - 1}{2} \rfloor$

$\lfloor \frac{\lfloor \frac{n-2}{2} \rfloor - 1}{2} \rfloor + 1, \dots, \lfloor \frac{n-2}{2} \rfloor$

* No of vertices of height "h",

$$\left\lfloor \frac{n-2^{h+1}}{2^{h+1}} \right\rfloor + 1, \dots, \left\lfloor \frac{n-2^h}{2^h} \right\rfloor$$

$$\leq \frac{h}{2^{h+1}}$$

$$\begin{aligned} * \sum_{h=0}^{\log_2 n} \frac{n}{2^{h+1}} \times O(h) &\Rightarrow O\left(\sum_{h=0}^{\log_2 n} \frac{n}{2^{h+1}} \times h\right) \\ &= O\left(n \cdot \sum_{h=0}^{\log_2 n} \frac{h}{2^{h+1}}\right) = O(n) \end{aligned}$$

02/02/26

Height(i):

Nodes of height 0: $n-1, n-2, \dots, \text{Parent}(n-1) + 1$

$$\left\lfloor \frac{n-2}{2} \right\rfloor + 1$$

$$\boxed{\text{Nodes of height 0: } n-1, n-2, \dots, \left\lfloor \frac{n-2}{2} \right\rfloor + 1}$$

Nodes of height 1: $\left\lfloor \frac{n-2}{2} \right\rfloor, \left\lfloor \frac{n-2}{2} \right\rfloor - 1, \dots, \text{Parent}\left(\left\lfloor \frac{n-2}{2} \right\rfloor\right) + 1$

$$\left\lfloor \frac{\left\lfloor \frac{n-2}{2} \right\rfloor - 1}{2} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{n-4}{2} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{n-4}{4} \right\rfloor$$

$$* \text{parent}(\text{parent}(n-1)) = \left\lfloor \frac{n-4}{4} \right\rfloor$$

$$\text{parent}^j(n-1) = \left\lfloor \frac{n-2^j}{2^j} \right\rfloor$$

$$\text{let, } \text{parent}^{j-1}(n-1) = \left\lfloor \frac{n-2^{j-1}}{2^{j-1}} \right\rfloor \text{ be True.}$$

$$\text{parent}(\text{parent}^{j-1}(n-1)) = \left\lfloor \frac{\left\lfloor \frac{n-2^{j-1}}{2^{j-1}} \right\rfloor - 1}{2} \right\rfloor = \left\lfloor \frac{\left\lfloor \frac{n-2^{j-1}-2^{j-1}}{2^{j-1}} \right\rfloor}{2} \right\rfloor$$

$$= \left\lfloor \frac{\left\lfloor \frac{n-2^j}{2^{j-1}} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{n-2^j}{2^j} \right\rfloor$$

Nodes of height h :

$$= \text{Parent}^{h+1}(n-1)+1, \dots, \text{Parent}^h(n-1)$$

$$= \left\lfloor \frac{n-2^{h+1}}{2^{h+1}} \right\rfloor + 1, \dots, \left\lfloor \frac{n-2^h}{2^h} \right\rfloor$$

$$\text{No. of nodes of height } h: \left\lfloor \frac{n-2^h}{2^h} \right\rfloor - \left\lfloor \frac{n-2^{h+1}}{2^{h+1}} \right\rfloor - 1 + 1$$

* no of nodes of height h

$$\leq \left\lfloor \frac{n-2^h}{2^h} - \frac{n-2^{h+1}}{2^{h+1}} \right\rfloor + 1$$

$$= \left\lfloor \frac{n}{2^{h+1}} \right\rfloor + 1$$

* Total running time of Build Heap:

$$= \sum_{h=0}^{\log_2 n} (\text{running time of Heapify on a node of height } h \times \text{no of nodes of height } h)$$

$$\text{Time spent by Heapify}(i) = O(h(i))$$

$$= \sum_{h=0}^{\log_2 n} c \times h \times \left(\left\lfloor \frac{n}{2^{h+1}} \right\rfloor + 1 \right) \leq \sum_{h=0}^{\log_2 n} c \times h \times \left(\frac{n}{2^{h+1}} + 1 \right)$$

$$\leq \underbrace{cn \sum_{h=0}^{\log_2 n} \frac{h}{2^{h+1}}}_{=1} + \sum_{h=0}^{\log_2 n} ch = O((\log_2 n)^2)$$

$$\leq cn + c(\log_2 n)^2 = O(n),$$

$$(\log_2 n)^2 = o(n).$$

EXTRACT_MIN (Array A): ^{length n}

$x \leftarrow A[0]$

Exchange $A[n-1] \leftrightarrow A[0]$

$n \leftarrow n-1$

Heapify(A, 0)

return x

FLOAT_UP (A, i):

if ($i == \text{root}$)

return

if ($A[i] < A[\text{parent}(i)]$) {

Exchange ($A[i] \leftrightarrow A[\text{parent}(i)]$)

FLOAT_UP (A, parent(i));

A "violation" is a pair ($i, \text{parent}(i)$) such that $A[i] < A[\text{parent}(i)]$.

* If ($i, \text{parent}(i)$) is a violation, then FLOATUP changes A so that the number of violations decreases.

* $A[i]$ is less than or equal to value of every node in the subtree rooted at $A[\text{left}(i)]$ and subtree rooted at $A[\text{right}(i)]$

* If ($i, \text{parent}(i)$) is the only violation in A, then FLOATUP(A, i) returns heap. and $\text{parent}(i)$ is less than or equal to every element in the subtrees $\text{left}(i) \& \text{right}(i)$; then FLOATUP returns a heap.

INSERT(A, val):

$n \leftarrow n+1$

$A[n-1] \leftarrow \text{val}$

FLOATUP (A, n-1)

Insertion Sort:

```

Sort (node* h) {
    node* t = Sort (h->next);
    insert (h, t);
}

```

```
Sort (node* h) {
```

```
    if (h == NULL) {
```

```
        return NULL;
```

```
    }
```

```
    t = Sort (h->next);
```

```
    return insert (h, t);
```

```
}
```

```
* Mergesort (A, p, q) {
    if (p == q) return;
    mid = (p+q)/2;
```

```
    mergesort (A, p, mid);
```

```
    mergesort (A, mid+1, q);
```

```
    merge (A, p, mid, q);
```

```
}
```

```
merge (A[], B[]) {
```

```
    i ← 0
```

```
    j ← 0
```

```
    k ← 0
```

```
    while (i < nA OR j < nB) {
```

```
        if (i == nA OR (j < nB AND B[j] < A[i])) {
```

```
            C[k] ← B[j];
```

```
            j ← j+1;
```

```
        }
```

```
    }
    else {
```

```
        C[k] ← A[i];
```

```
        i ← i+1;
```

```
        k ← k+1;
```

```
}
```

n_A, n_B

$A = A[0], \dots, A[n_A-1]$

$B = B[0], \dots, B[n_B-1]$

A & B are sorted in non-decreasing order.

LOOP INVARIANT:

~~At the start of loop~~, When while termination condition is checked:

$(C[0], \dots, C[k-1])$ is ~~union~~ union with repetition of
 $(A[0], \dots, A[i-1])$ and $(B[0], \dots, B[j-1])$ in sorted order.

→ In the last execution of first statement of the while loop,

$$i = n_A, j = n_B$$

$C[0], C[1], \dots, C[k-1]$ is union with repetition of $(A[0], \dots, A[n_A-1])$
+ $B[0], B[1], \dots, B[n_B-1]$.

→ i_t → value of "i" at t^{th} iteration.

* For any variable v ,

v_t is the value of v when the while termination condition is checked for t -th time.

$$\begin{array}{l|l} i_1 = 0 & C[0], \dots, C[k_1-1] \\ j_1 = 0 & A[0], \dots, A[i_1-1] \\ k_1 = 0 & B[0], \dots, B[j_1-1] \end{array}$$

* Invariant holds for $t=1$.

Suppose that invariant holds for some $t \geq 1$,

To show that invariant holds for $t+1$.

$$(C[0], \dots, C[k_t-1]) = (A[0], \dots, A[i_t-1]) \cup (B[0], \dots, B[j_t-1])$$

Case 1: The 'if' part was executed in the t -th iteration of the while loop.

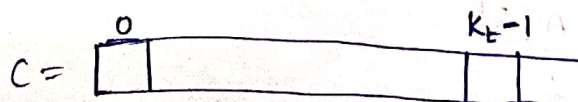
Either $i_t = n_A$ or $(j_t < n_B \wedge B[j_t] < A[i_t])$

$$C[k_t] \leftarrow B[j_t]$$

$$k_{t+1} = k_t + 1$$

$$i_{t+1} = i_t + 1$$

$$j_{t+1} = j_t + 1$$



$C[0], \dots, C[k_t-1]$ is the union of $A[0], \dots, A[i_t-1]$ and $B[0], \dots, B[j_t-1]$.

$$C[k_t] = B[j_t] \\ = B[j_{t+1}-1]$$

09/02/26

Merge(A, B) {

$i \leftarrow 0, j \leftarrow 0, k \leftarrow 0$

while ($i < n_A$ or $j < n_B$) {

if ($i = n_A$ OR ($j < n_B$ & $B[j] \leq A[i]$))

$C[k] \leftarrow B[j]$

$k \leftarrow k+1; j \leftarrow j+1;$

else :

$C[k] \leftarrow A[i];$

$k \leftarrow k+1, i \leftarrow i+1;$

}

- ① Loop invariant (satisfied each time the while condition is checked):
 $C[0], \dots, C[k-1]$ is union of $A[0], A[1], \dots, A[i-1]$ & $B[0], B[1], \dots, B[j-1]$ in sorted order.

Suppose that

$C[0], \dots, C[k_t-1]$ is union of $A[0], A[1], \dots, A[i_t-1]$ and $B[0], \dots, B[j_t-1]$ in sorted order.

In t^{th} iteration of while loop, To show $C[0], \dots, C[k_{t+1}-1]$ is union of $A[0], A[1], \dots, A[i_{t+1}-1]$ & $B[0], \dots, B[j_{t+1}-1]$ in sorted order.

Case 1: Execution went inside the if-block

$$k_{t+1} = k_t + 1, i_{t+1} = i_t, j_{t+1} = j_t + 1$$

$$C[k_t] = C[k_{t+1}-1] = B[j_t] = B[j_{t+1}-1]$$

$C[0], \dots, C[k_{t+1}-1]$ is union of $\underbrace{C[0], \dots, C[k_t-1]}_{(A[0], \dots, A[i_t-1]) \cup (B[0], \dots, B[j_t-1])}$ and $B[j_t], \dots, B[j_{t+1}-1]$.

$\therefore C[0], \dots, C[k_{t+1}-1]$ is union of $A[0], \dots, A[j_t-1]$ and $B[0], \dots, B[j_{t+1}-1]$.

② If $i_t < n_A$ then $A[i_t]$ is greater than or equal to every element in $C[0], \dots, C[k_t-1]$.

③ If $j_t < n_B$ then $B[j_t]$ is greater than or equal to every element in $C[0], \dots, C[k_t-1]$.

* $B[j_t] \geq C[h], 0 \leq h \leq k_t-1$

$B[j_{t+1}] \geq B[j_t] \Rightarrow B[j_{t+1}] \geq C[h], 0 \leq h \leq \underbrace{k_t}_{j_{t+1}} - 1$

We have to show that,

$$A[i_{t+1}] = A[i_t]$$

$$A[i_t] \leq C[h], 0 \leq h \leq k_t-1$$

$$A[i_{t+1}] \leq C[h]; 0 \leq h \leq k_t-1$$

$$A[i_t] \geq C[k_{t+1}-1] \text{ else } [i_t, \dots, [k_t]$$

$$B[j_t] = C[k_t]$$

$$i_t < n_A \Rightarrow j_t < n_B \text{ and } B[j_t] < A[i_t]$$

Inductively, $A[i_t] \geq C[h], 0 \leq h \leq k_t-1$.

$$A[i_t] = A[i_{t+1}] \geq C[h], 0 \leq h \leq k_t-1.$$

$B[j_t] = C[k_t]$, because $B[j_t]$ is put in $C[k_t]$.

$$\neq B[j_t] < A[i_t]$$

$$A[i_t] \geq C[h], \forall h \text{ in } 0 \leq h \leq \underbrace{k_t}_{j_{t+1}} - 1.$$

Termination: $i = n_A$ and $j = n_B$

→ C is the union of $A[0], \dots, A[n_A-1]$ and $B[0], \dots, B[n_B-1]$ in sorted order.

Quick Sort

PARTITION(int $A[]$) {

QuickSort(int $A[]$, int p , int q) {

if ($q - p \leq 1$)
return;

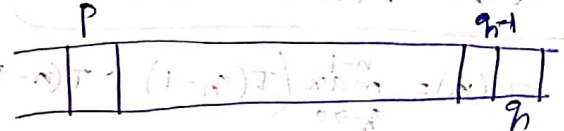
{

$i \leftarrow \text{PARTITION}(A, p, q)$;

QuickSort(A, p, i);

QuickSort($A, i+1, q$);

}



nt PARTITION(int $A[]$, p , q) {



$x \leftarrow A[q]$

$i \leftarrow 0$

$b \leftarrow 0$

while ($i < q$) {

if ($A[i] \leq x$) {

swap($A[i], A[b]$);

$b \leftarrow b+1$

}

$i++$;

}

$b++$;

swap($A[b], A[x]$);

return b ;

}



Modifies array $A[p] \dots A[q]$ & returns a value b such that

$$p \leq b \leq q-1 \text{ \& } A[b] \leq A[j], 0 \leq j < b$$

$$A[j] \geq A[b], b < j \leq q-1$$

∴ * $b \leq i$

→ * j such that $0 \leq j < b, A[j] \leq x$

② * j such that $b \leq j < i, A[j] > x$

* $T(n)$ - time taken by Quick Sort on any array of n -elements in worst case.

③

~~$$T(n) = T(n-1) + T(n-n)$$~~

* * $T(n) = \max_{q \rightarrow 0}^{n-1} (T(q-1) + T(n-q)) + O(n)$

$$T(n) \leq cn^2$$

To
$$T(n) \leq \max_{q \rightarrow 0}^{n-1} (cq^2 + c(n-q-1)^2) + O(n)$$

$$= \max_{q \rightarrow 0}^{n-1} (cq^2 + c(n-q-1)^2) + O(n)$$

~~$$f(q) = \max_{q \rightarrow 0}^{n-1} 2cq^2 + cn^2 + 2cn + c - 2cnq - 2cn$$~~

$$f'(q) = 4cq + 2c - 2cn \quad \therefore cq^2 + c(n-q-1)^2 \rightarrow \text{max at } q=0 \text{ (or) } q=n-1$$

$$f''(q) = 4c$$

$$T(n) \leq c(n-1)^2 + O(n)$$

$$T(n) \leq cn^2 - 2cn + c + O(n)$$

For an appropriate value of c , for large enough values of n , we have

$$T(n) \leq cn^2$$

∴ Worst Case Time Complexity of Quick Sort = $O(n^2)$